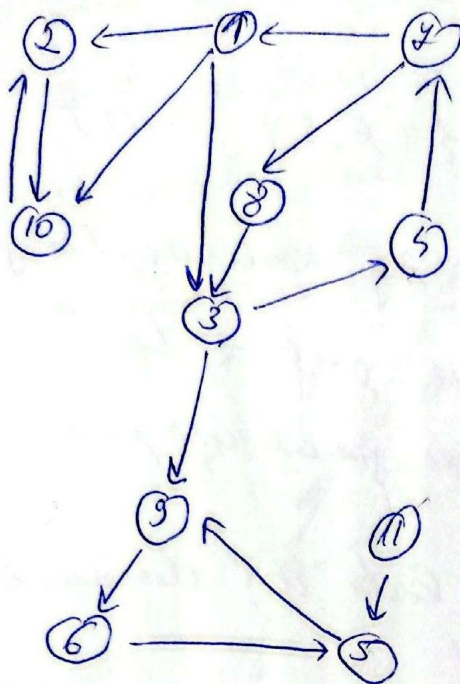




1) are o sortare topologica?

→ un graf are o sortare topologica doar dacă este aciclic (DAG)

in grafal nostru sunt cicluri:

1 - 3 - 5 - 4

4 - 8 - 3 - 4

9 - 6 - 5

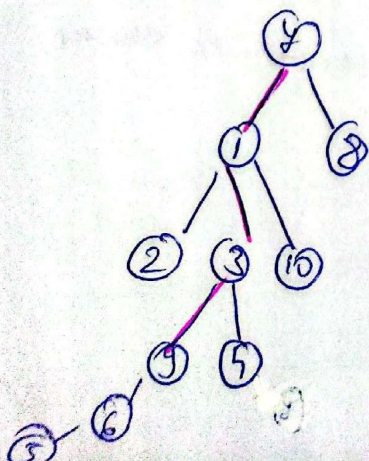
⇒ nu are o sortare topologica

BFS 2) $lf(7)$ - parcurgere în latime

7 1 8 2 3 10 9 4 6 5

→ se ia nodul x se parcurg vecinii lui și dacă sunt nevizitați se adaugă în coadă, apoi, se merge la următorul elem. din coadă și se repetă pașii până nu se mai poate accesa niciun nod.

7 1 8 2 3 10 9 4 6 5



arborele bfs

pt a determina distanța min se verifică dacă noul vecin este destinația căutată și se reconstruiește în ajutorul unui vector y de întregi
 $d(7, 9) = 3$

3) Care sunt componentele tare conexe ale grafului...

Adăugând un arc a.b. $\Rightarrow$ o c.t.c. mai mare.

CTC = $\{\{2, 10\}, \{1, 3, 4, 7, 8\}, \{9, 6, 5\}, \{11\}\}$
(SCC)

CTC se pot obține prin Alg. Kosaraju/Tarjan.

A. Kosaraju - pași: 1) DFS pe graf și țin minte
timpul de finalizare pentru fiecare
nod

2) sortează lista de descendenți

3) fac G^t

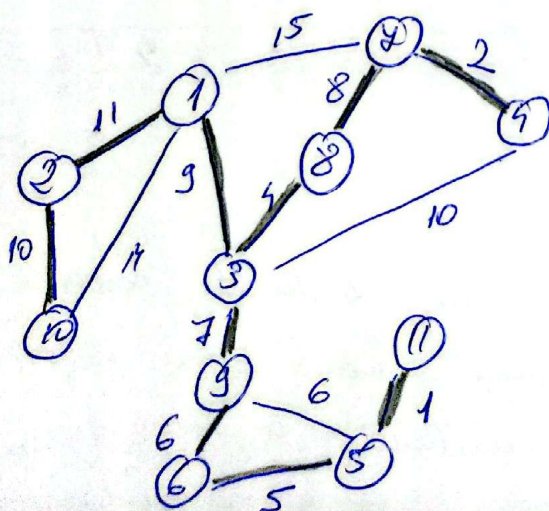
4) fac DFS din fiecare nod care nu
a fost vizitat din lista de mai
sus pe graful transpus

ad: $3 \nrightarrow 10$, $10 \rightarrow 3 = 7$ noduri. $\times$
 $5 \rightarrow 3 = 3$ noduri. $\checkmark$

Adăugăm muchia (5, 3).

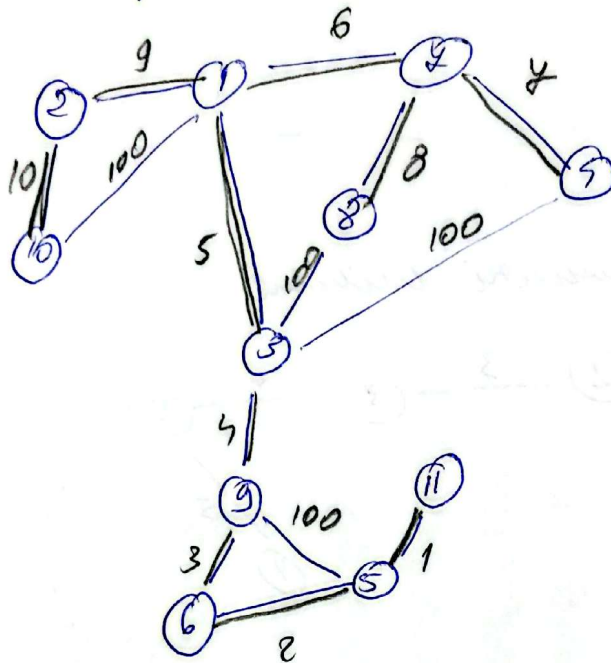
4) grafurile asociate - puncte ponderale a. d. graful
are cel puțin un unic APM

APM - se determină cu
Kruskal sau Prim

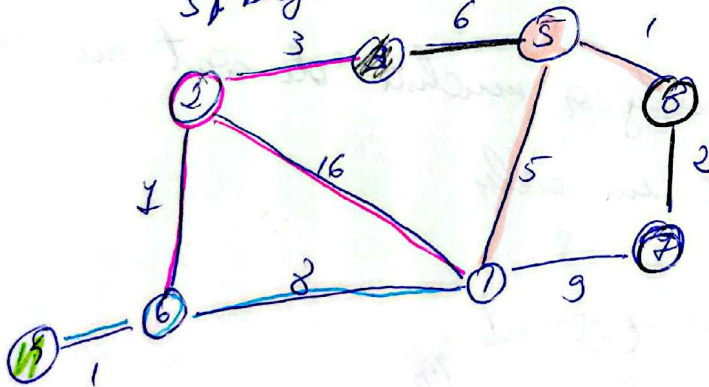


*Truc : MST unic, dar cu ponderi egale

facem toate muchiile din MST să aibă ponderi
strict crescătoare și mici, iar celelalte muchii să
aibă ponderi mult mai mari, cu egalități între ele.



5) Dijkstra din u₁ 2



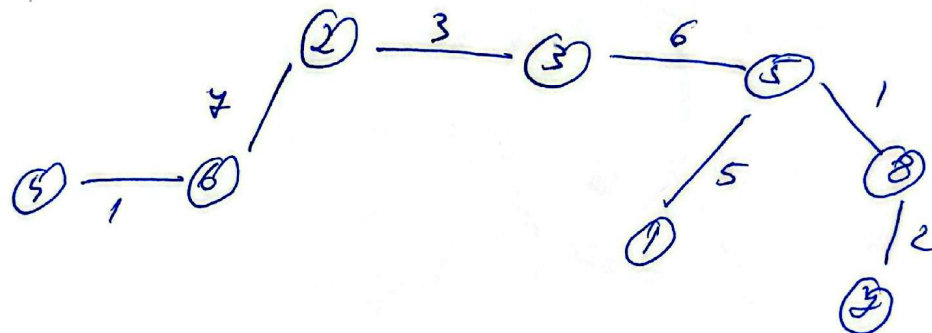
→ alg. calcularea drumului
minim de la sursă 2
la toate celelalte
u_j, folosindu-se
la fiecare pas de
relaxarea axelor
care pleacă din nodul
respectiv

d/tata	1	2	3	4	5	6	7	8
	[∞/0	0/0	∞/0	∞/0	∞/0	∞/0	∞/0	∞/0]
	16/2		3/2		3+6 9	7/2		
m=3	16/2	0/0	-	∞/0	9/3	7/2	∞/0	∞/0
m=6	15/6	0/0	-	8/6	9/3	-	∞/0	∞/0
m=5	15/6	0/0	-	-	9/3	-	∞/0	∞/0

↳ nu se actualizează nimic

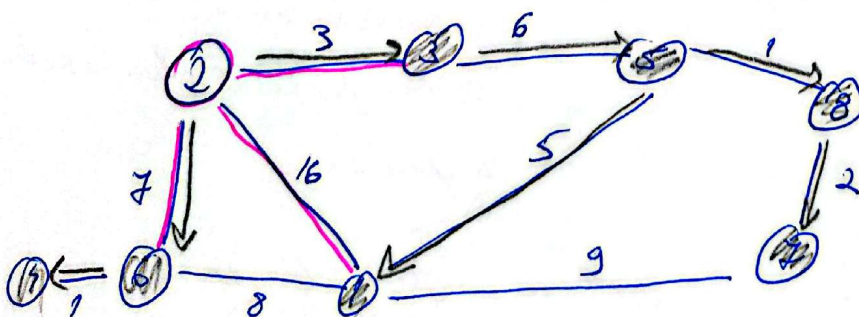
	1	2	3	4	5	6	7	8
$n=5$	14/5	0/0	—	—	—	—	00/0	10/5
$n=8$	14/5	0/0	—	—	—	—	12/8	—
$n=7$	14/5	0/0	—	—	—	—	—	—
$n=1$	—	—	—	—	—	—	—	—

graful de drumuri minime



6) Alg. lui Prim din 2

- pt APM
- la fiecare pas se alege muchia de cost minim care nu formează un ciclu



	1	2	3	4	5	6	7	8
d/tota	$\infty/0$	0/0	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$	$\infty/0$
	16/2	0/0	<u>3/2</u>			7/2		
$n=3$	16/2	0/0	-	$\infty/0$	<u>6/3</u>	7/2	$\infty/0$	$\infty/0$

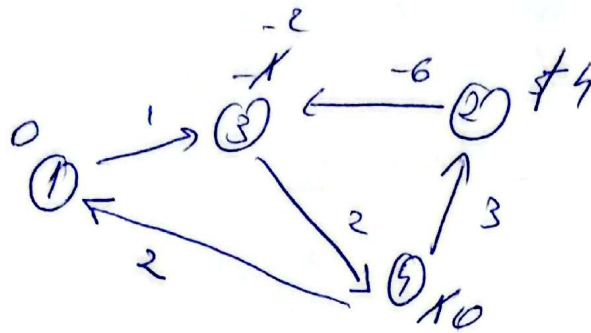
→ pun doar costul muchiei de la nodul anterior până la el (cu mai ordine ca la Dijkstra) → aici mă interesează doar costul fiecărei muchii.

$n=5$	5/5	0/0	-	$\infty/0$	-	7/2	$\infty/0$	<u>1/5</u>
$n=8$	5/5	0/0	-	$\infty/0$	-	7/2	<u>2/8</u>	-
$n=4$	<u>5/5</u>	0/0	-	$\infty/0$	-	7/2	-	-
$n=1$	-	0/0	-	<u>$\infty/0$</u>	-	7/2	-	-
	-	0/0	-	$\infty/0$	-	<u>7/2</u>	-	-
$n=6$	-	0/0	-	<u>4/6</u>	-	-	-	-

$$\text{costul APM} = 3 + 6 + 1 + 2 + 5 + 7 + 1 = 25$$

⑦) Bellman - Ford → calcularea drumului minim de la un nod sursă la toate celelalte noduri, chiar dacă muchiile au, și ponderi negative, prin relaxarea la fiecare pas și

a tuturor muchiilor



// detectare ciclu negativ $\rightarrow$ mai face o relaxare

pentru fiecare $uv \in E$ execută

dacă $d[u] + w(u, v) < d[v]$ atunci

scrie 'ciclu negativ'

$\hookrightarrow$ Se mai face încă o relaxare și dacă
încă se mai actualizează valorile înseamnă
că este un ciclu negativ.

$$E = \{(1, 3), (4, 1), (4, 2), (2, 3), (3, 4)\}$$

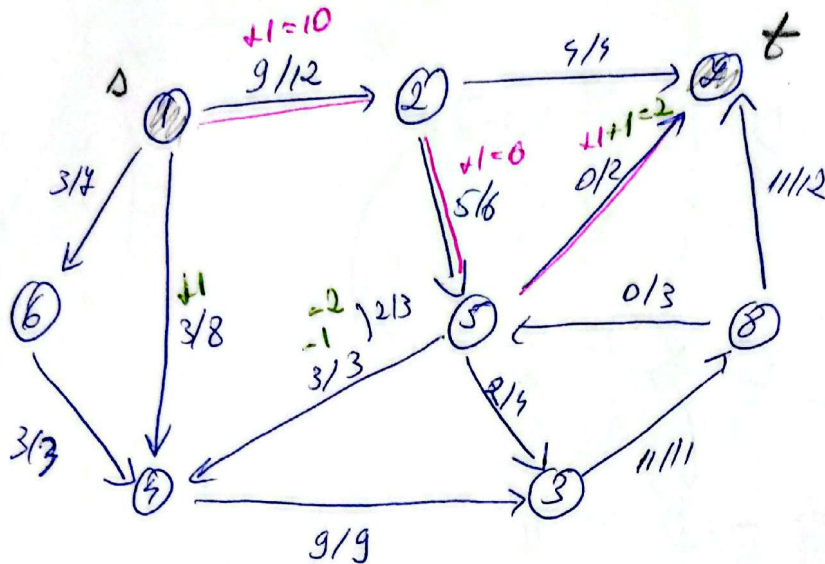
nod	1	2	3	4
d	0	5	-1	1
		4	-2	0

$\Rightarrow$ încă se actualizează deci e un ciclu

2 - 3 - 4

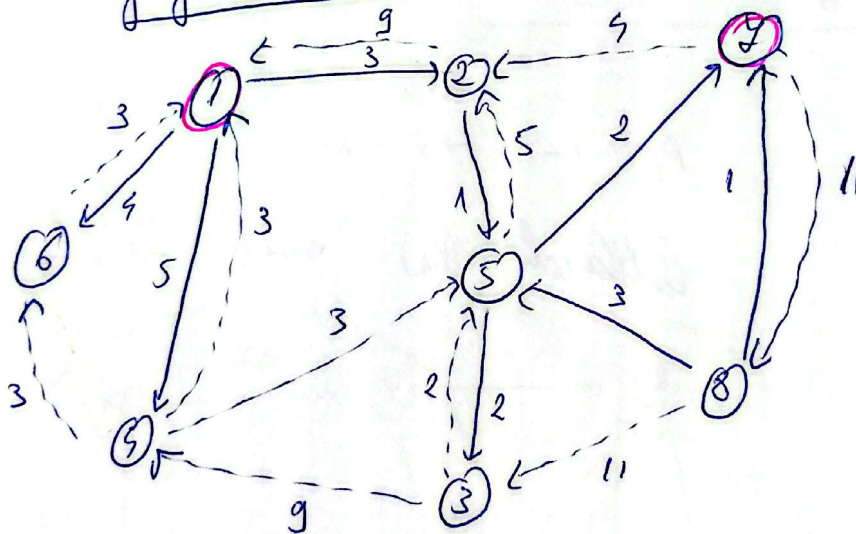
8) Fluxul - f/c

$$s=1, t=7$$



val flux initial : $val(f) = 3 + 3 + 9 = 15$

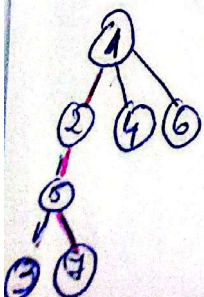
graf residual



I 1) Edmonds - Karp (BFS pe grafel residual)

BFS						
1	2	4	6	5	3	7

$$P_1: 1 \rightarrow 2 \rightarrow 5 \rightarrow 7$$

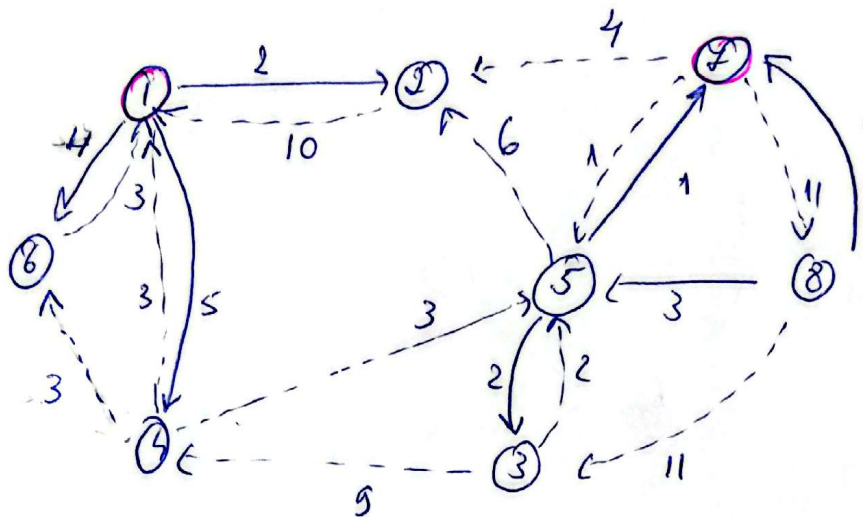


bottleneck

$$c(P_1) = \min(3, 1, 2) = 1$$

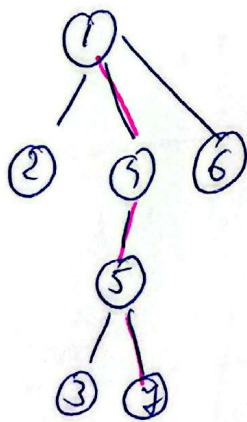
2) Actualizare flux

3) Refacere residual



II) BFS

1	2	4	6	5	3	7
.	.	.	.	.	.	.

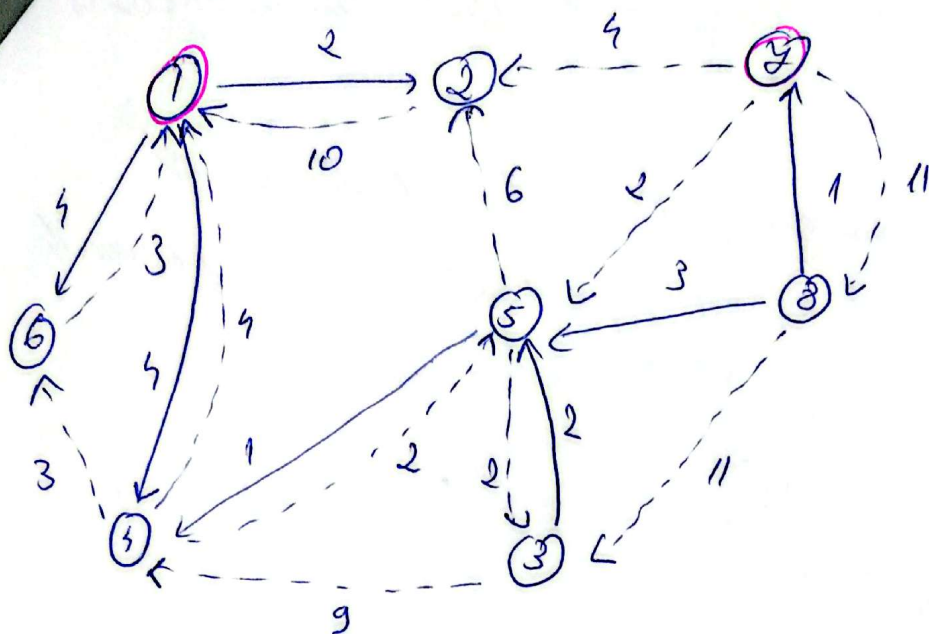


$P_2 : 1 \rightarrow 4 \rightarrow 5 \rightarrow 7$

bottleneck: $i(P_2) = \min(3, 3, \underline{1}) = \underline{1}$

2) Actualizare flux

3) Refacere graf residual



II

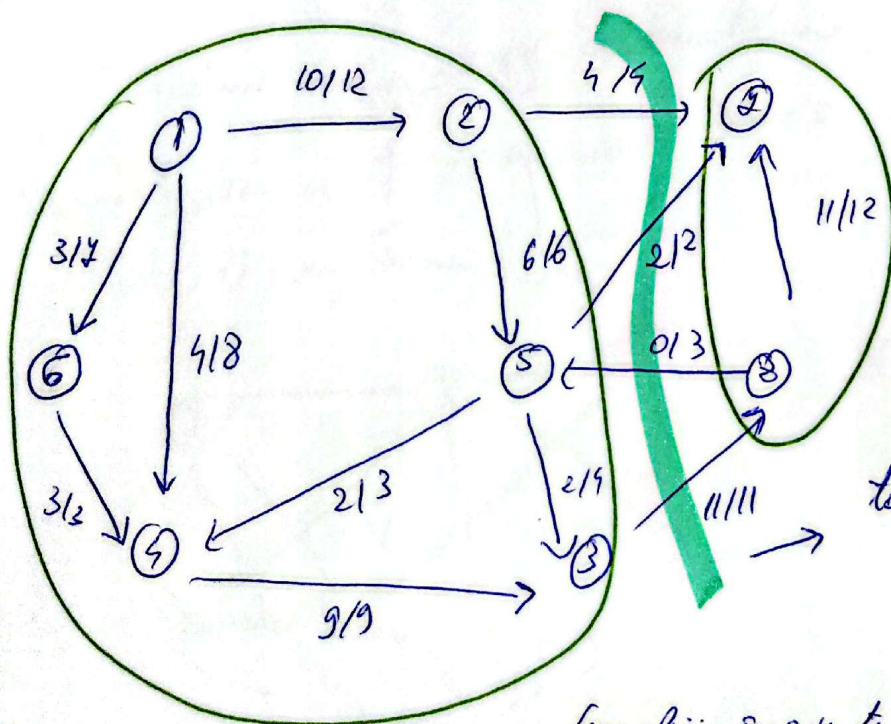
BFS					
1	2	4	6	5	3

→ nu mai găsește drum la 4

$$\text{val f!} = 3 + 4 + 10 = 17$$

$$\Rightarrow X = \{1, 2, 4, 6, 5, 3\}$$

$$Y = \{3, 4, 8\}$$

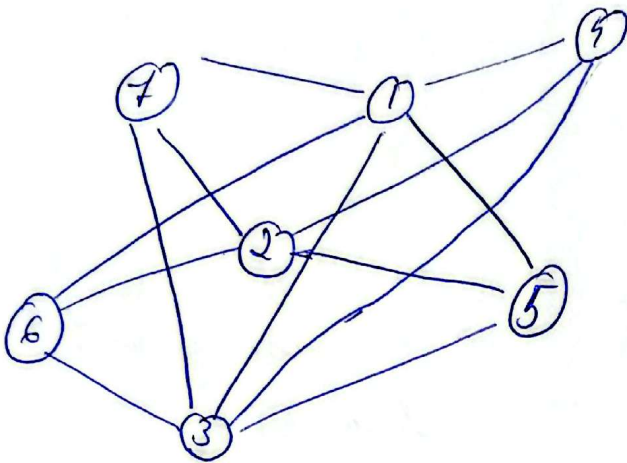


tăietura min =
 $4 + 2 + 11 = 17$

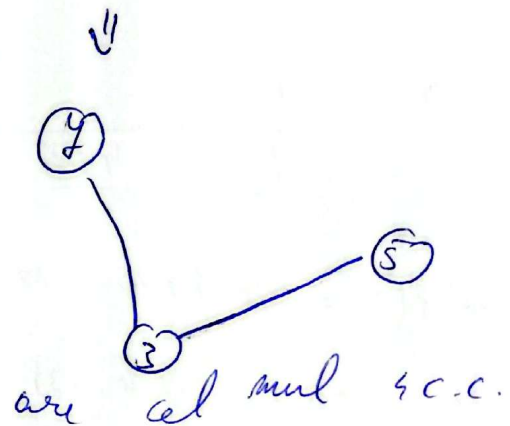
(muchii maxime) arcel directe: (2, 4), (5, 4), (3, 8)

9) a) $G = (V, E)$ un graf neorientat hamiltonian
conex.

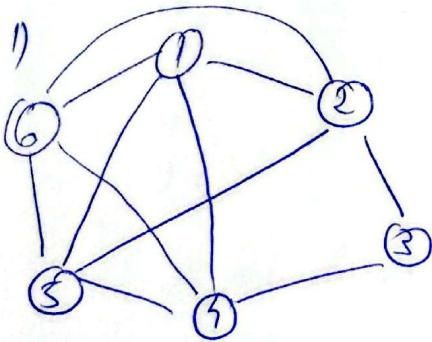
$X \subset V \rightarrow$ subgraful obținut din $V - X$
are cel mult $|X|$ componente
conex



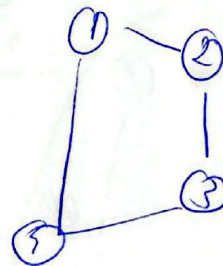
$$X = \{1, 2, 4, 6\} \rightarrow 4$$



Pe caz general

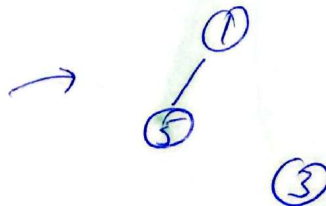
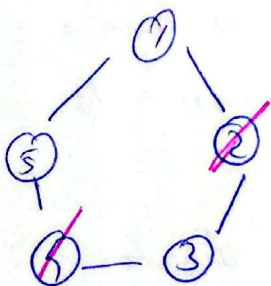


$$X = \{6, 5\}$$



$\rightarrow$ cel mult 2 c.c.

2)



$$X = 2$$

cel mult 2 c.c.

Fie C un ciclu hamiltonian care trece o dată prin toate v .

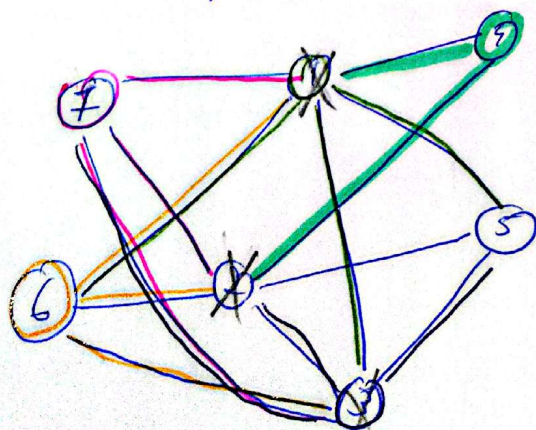
Dem.: Dacă eliminăm v din C .

- Un ciclu după eliminarea a $|X|$ v , se "rup" în cel mult $|X|$ bucăți. Dacă graful $C-X$ are cel mult $|X|$ componente conexe.

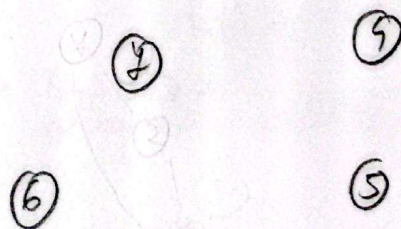
- Dar C este subgraf al lui G . Eliminand aceiași v , $G-X$ nu poate avea mai mult c.c. decât $C-X$

b) Folosind a), arătăm că graful nu e hamiltonian.

Pn că graful e conex și hamiltonian.
(trebuie să găsim o mulțime X de noduri care dacă sunt eliminate din G dau mai mult de $|X|$ componente conexe).



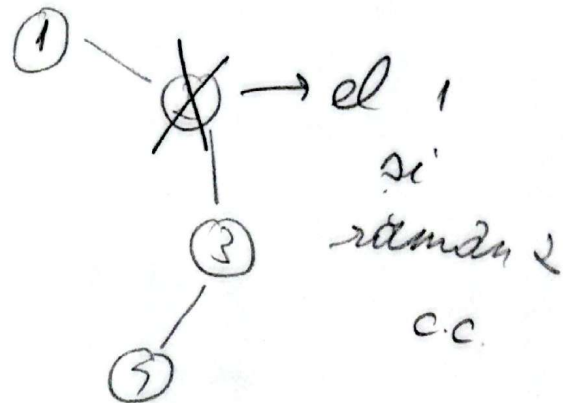
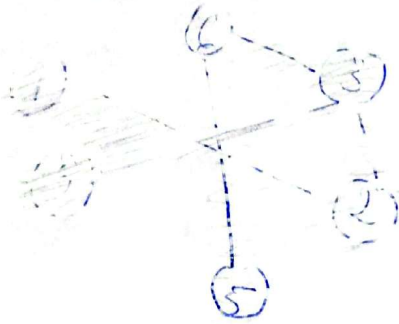
$$X = \{1, 2, 3\} \rightarrow |X| = 3$$



dar am 4 c.c.

$\rightarrow$ contradicție

c) scrie ex de un graf neorientat conex care
nu are proprietatea 91.



10.

$$m \rightarrow w_1$$

$$n \rightarrow w_2$$

$$c[i][0] = c \cdot w_2 \text{ (stergere)} \downarrow$$

$$c[0][j] = j \cdot w_2 \text{ (inserare)}$$

$$\begin{cases} c[i-1][j] - \text{stergere} \\ c[i-1][j] + w_2 \end{cases} \rightarrow$$

$$\begin{cases} c[i-1][j-1] - \text{modificare} \\ c[i-1][j-1] + w_1 \end{cases}$$

$$\begin{cases} c[i][j-1] - \text{inserare} \\ \Rightarrow \text{stergere in sens invers} \end{cases}$$

$$\Rightarrow \text{cost: } w_2$$

$$\bullet A[i] = B[j]$$

$$c[i][j] = c[i-1][j-1]$$

$$\bullet A[i] \neq B[j]$$

$$c[i][j] = \min \{ c[i-1][j] + w_2, c[i-1][j-1] + w_1, c[i][j-1] + w_2 \}$$

11. 6 - colorare , n nr noduri graf

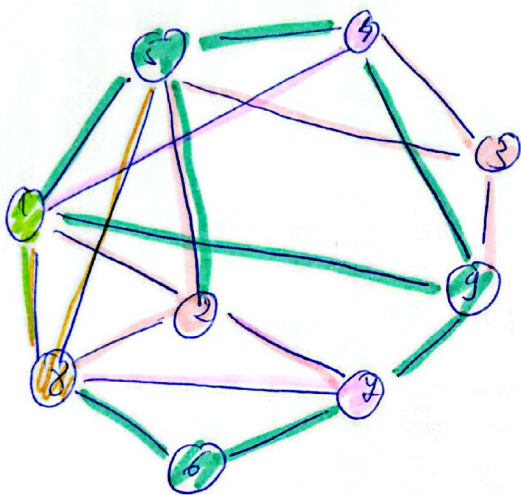
Pasi: 1. dacă $n \leq 6$ - colorăm fiecare nod diferit

2. dacă $n > 6 \rightarrow$ aleg un nod x cu $grad \leq 5$

$\rightarrow$ îl eliminăm din graf

$\rightarrow$ graf mai mic îl colorăm recursiv

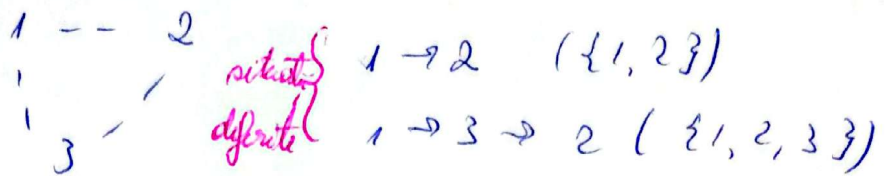
$\rightarrow$ adăugăm x la loc și îl colorăm dif de ceilalți.



12. în graf orientat, luna poate porni din orice vârf, poate repara vârfuri, dar vrea un drum cu număr minim de stări și drum care trece prin cel puțin p stări distincte.

Rezoluare - Facem un graf de stări

(în ce stare suntem, stăruim cu ce stări am văzut)



$\rightarrow (2, \{1, 2\}), (2, \{1, 2, 3\}), (3, \{1, 3\})$

v = cf curent
 $S \subseteq V$ = multimea cf distinct vizitate pana acum

Transitie: daca exista arc $v \rightarrow u$, atunci:

$$(v, S) \rightarrow (u, S \cup \{u\})$$

Cost: fiecare trecere adauga o "stare" in drum (adica +1 la lungime in cf/pasi)

Start: poate porni din orice nod

Tinta: orice stare (u, S) cu $|S| \geq p$

Algoritmul: - rulăm BFS in grafurile de stări
 - păstrăm tabloul $[(u, S)]$ ca să reconstruim drumul

Corectitudinea: $\rightarrow$ BFS găsește cel mai scurt nr. de pași in grafurile de stări.

$\rightarrow$ Orice drum real al lui u corespunde unei succesiuni de stări (u, S) , iar invers

Deci prima dată când BFS ajunge într-o stare cu $|S| \geq p$ avem drumul minim

$$O(m \cdot 2^n)$$